Detecting burst events from repost trees

Some people share the original post. Others pick up the message from an earlier repost. Connect these steps into a tree and ask a model to classify the cascade as normal or bursty.

20,648	82.25%	0.794
Cascade graphs	Test accuracy	Test macro F1

Describe the spread

How often is the original post shared directly? How many steps does the message travel?

Classify the whole tree

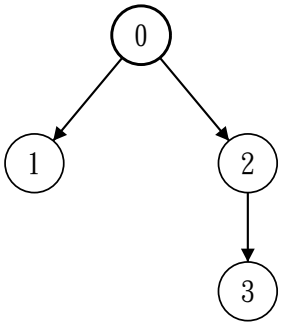
Use the tree shape and post vectors to score how likely the cascade is to be bursty.

Test on unseen trees

Measure how well the model classifies trees it has never seen.

Read a repost tree

A node is a post. A directed edge points from the source post to the repost.



Post 0 is the original. Posts 1 and 2 repost it, and post 3 reposts post 2.

edge_index	The rows are [0, 0, 2] and [1, 2, 3].
x	Each post has a vector of 32 numbers.
label	The graph label is 0 for normal or 1 for burst.

Dataset splits

Split	Graphs	Normal	Burst
Training	14,453	9,633	4,820
Validation	3,097	2,064	1,033
Test	3,098	2,065	1,033

Average shape in the training set

Graph feature	Normal	Burst
Number of posts	42.44	42.70
Maximum depth	4.90	4.10
Share of posts on the widest level	39.9%	51.1%
Share of reposts directly from the original	38.2%	51.6%
Share of posts with no downstream reposts	63.0%	69.5%

The two classes are similar in size. Burst cascades have more direct reposts of the original and a greater share of posts on the widest level. Each tree has N-1 edges, so its directed density is 1/N and reflects only its size.

Classify the whole tree

Logistic regression turns a graph into a single row of numbers. Seven features describe its shape. Averaging the 32-dimensional vectors across all posts gives a second description. Combining the two produces 39 inputs.

The graph isomorphism network, or GIN, lets neighboring posts exchange information twice. It averages the resulting representations and scores the whole tree. The validation set selects the training epoch. The test set measures performance.

Test set 3,098 graphs

Method	Accuracy	Macro F1	Burst recall	ROC-AUC
Always predict normal	0.667	0.400	0.000	0.500
Post count only	0.667	0.400	0.000	0.488
Graph structure	0.731	0.664	0.428	0.766
Mean post vector	0.777	0.733	0.556	0.835
Structure and mean vector	0.822	0.794	0.674	0.886
GIN	0.796	0.760	0.612	0.850

Accuracy counts correct predictions across all samples. Macro F1 gives both classes equal weight by averaging their F1 scores. Predicting normal every time misses every burst cascade.

Where the best model made mistakes

Actual class	Predicted normal	Predicted burst
Normal	1,852	213
Burst	337	696

Combining structure and mean vectors caught 696 of 1,033 burst cascades, for a burst recall of 67.4%.

Run the experiments

Start in the repository root with Python 3.10 or later.

```
python -m pip install -r requirements.txt
cd experiments/weibo_cascades
python generate_dataset.py --out data
python run_experiments.py
```

If you already have the three .pt files, skip generation and pass --data-dir followed by the folder containing the files.

Find the outputs

File	What it contains
metrics.json	Event metrics and training history
event_test_predictions.csv	Event labels and model scores
gin_model.pt	Model weights selected on validation

How to interpret the results

These scores measure classification on complete trees in a simulation. Timestamps are needed to measure actual spreading speed. To test early warnings, choose a cutoff and give the model only the reposts observed before it.